Introduccion

Este informe analiza los datos de clientes de TelecomX LATAM para identificar los factores que influyen en la cancelación (churn) de servicios. Los datos provienen del archivo "datos_limpiados.csv", que contiene información sobre 7,032 clientes, incluyendo variables demográficas, de servicio y financieras. El churn afecta aproximadamente al 26.58% de los clientes (media de la variable Churn). El análisis se basa en:

Variables seleccionadas: Género, edad (SeniorCitizen), estado civil (Partner), dependientes, antigüedad (tenure), servicios (PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies), tipo de contrato, facturación sin papel y método de pago, cargos mensuales y totales, y cuentas diarias. Modelos de machine learning: Se evaluaron Logistic Regression, Random Forest, Gradient Boosting para predecir churn. Estos modelos ayudan a cuantificar la importancia de las variables y evaluar el rendimiento predictivo.

El objetivo es destacar los factores clave de churn y proponer estrategias de retención. El análisis se realizó utilizando Python con bibliotecas como pandas, scikit-learn y matplotlib para procesamiento y modelado.

Importacion de Datos

```
import pandas as pd

df = pd.read_json('/content/datos_limpiados.json')

df.drop(columns='customerID',inplace=True)
df[['OnlineSecurity','OnlineBackup','DeviceProtection','TechSupport','StreamingTV','StreamingMovies']] = df[['OnlineSecurity','OnlineBackup','DeviceProtection','TechSupport','StreamingTV','StreamingMovies']].replace('No internet service',0)
df['MultipleLines']=df['MultipleLines'].replace('No phone service',0)
df['gender']=df['gender'].replace('Male',0)
df['gender']=df['gender'].replace('Female',1)
```

```
/tmp/ipykernel_459/3154146476.py:6: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option(
  df[['OnlineSecurity','OnlineBackup','DeviceProtection','TechSupport','StreamingTV','StreamingMovies']] = df[['OnlineSecurity','OnlineBackup','DeviceProtection','TechSupport','StreamingTV','StreamingMovies']].replace('No internet service',0)
/tmp/ipykernel_459/3154146476.py:7: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option(
  df['MultipleLines']=df['MultipleLines'].replace('No phone service',0)
/tmp/ipykernel_459/3154146476.py:9: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option(
  df['gender']=df['gender'].replace('Female',1)
```

```
columnas_str = ['InternetService','PaymentMethod','Contract']
```

```
df[columnas_str] = df[columnas_str].astype(str)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 7032 entries, 0 to 7266
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Churn             7032 non-null   int64
 1   gender            7032 non-null   int64
 2   SeniorCitizen     7032 non-null   int64
 3   Partner           7032 non-null   int64
 4   Dependents        7032 non-null   int64
 5   tenure            7032 non-null   int64
 6   PhoneService      7032 non-null   int64
 7   MultipleLines     7032 non-null   int64
 8   InternetService   7032 non-null   object
 9   OnlineSecurity    7032 non-null   int64
 10  OnlineBackup      7032 non-null   int64
 11  DeviceProtection  7032 non-null   int64
 12  TechSupport       7032 non-null   int64
 13  StreamingTV       7032 non-null   int64
 14  StreamingMovies   7032 non-null   int64
 15  Contract          7032 non-null   object
 16  PaperlessBilling  7032 non-null   int64
```

```
 17  PaymentMethod     7032 non-null   object
 18  Charges.Monthly   7032 non-null   float64
 19  Charges.Total     7032 non-null   float64
 20  Cuentas_Diarias   7032 non-null   float64
dtypes: float64(3), int64(15), object(3)
memory usage: 1.2+ MB
```

```
df.describe()
```

	Churn	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	OnlineSecurity	OnlineBackup	DeviceProtection	TechSupport	StreamingTV	StreamingMovies	PaperlessBilling	Charges.Monthly	Charges.Total	Cuentas_Diarias
count	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000	7032.000000
mean	0.265785	0.495307	0.162400	0.482509	0.298493	32.421786	0.903299	0.421928	0.286547	0.344852	0.343857	0.290102	0.384386	0.388367	0.592719	64.798208	2283.300441	2.131520
std	0.441782	0.500014	0.368844	0.499729	0.457629	24.545260	0.295571	0.493902	0.452180	0.475354	0.475028	0.453842	0.486484	0.487414	0.491363	30.085974	2266.771362	0.989670
min	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	18.250000	18.800000	0.600329
25%	0.000000	0.000000	0.000000	0.000000	0.000000	9.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	35.587500	401.450000	1.170641
50%	0.000000	0.000000	0.000000	0.000000	0.000000	29.000000	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	70.350000	1397.475000	2.314145
75%	1.000000	1.000000	0.000000	1.000000	1.000000	55.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	89.862500	3794.737500	2.956003
max	1.000000	1.000000	1.000000	1.000000	1.000000	72.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	118.750000	8684.800000	3.906250

```
X = df.drop(columns='Churn')
y = df['Churn']
```

Encoding

```
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
```

```
columnas = X.columns
one_hot = make_column_transformer((OneHotEncoder(drop='if_binary'),['InternetService','PaymentMethod','Contract']),remainder='passthrough',sparse_threshold=0,force_int_remainder_cols=False)
```

```
X = one_hot.fit_transform(X)
```

```
one_hot.get_feature_names_out(columnas)
```

```
array(['onehotencoder__InternetService_0',
       'onehotencoder__InternetService_DSL',
       'onehotencoder__InternetService_Fiber optic',
       'onehotencoder__PaymentMethod_Bank transfer (automatic)',
       'onehotencoder__PaymentMethod_Credit card (automatic)',
       'onehotencoder__PaymentMethod_Electronic check',
       'onehotencoder__PaymentMethod_Mailed check',
       'onehotencoder__Contract_Month-to-month',
       'onehotencoder__Contract_One year',
       'onehotencoder__Contract_Two year', 'remainder__gender',
       'remainder__SeniorCitizen', 'remainder__Partner',
       'remainder__Dependents', 'remainder__tenure',
       'remainder__PhoneService', 'remainder__MultipleLines',
       'remainder__OnlineSecurity', 'remainder__OnlineBackup',
       'remainder__DeviceProtection', 'remainder__TechSupport',
       'remainder__StreamingTV', 'remainder__StreamingMovies',
       'remainder__PaperlessBilling', 'remainder__Charges.Monthly',
       'remainder__Charges.Total', 'remainder__Cuentas_Diarias'],
      dtype=object)
```

```
df_encoding = pd.DataFrame(X,columns = one_hot.get_feature_names_out(columnas))
```

```
y.value_counts()
```

Churn	count
0	5163
1	1869

dtype: int64

Balanceo de cuentas

```
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import KFold, cross_validate
from sklearn.tree import DecisionTreeClassifier
```

```
oversample = SMOTE()
```

```
X_balanceada, y_balanceada = oversample.fit_resample(X, y)
```

```
y_balanceada.value_counts()
```

Churn	count
0	5163
1	5163

dtype: int64

Matriz correlacion

```
df_encoding['Churn']= y
```

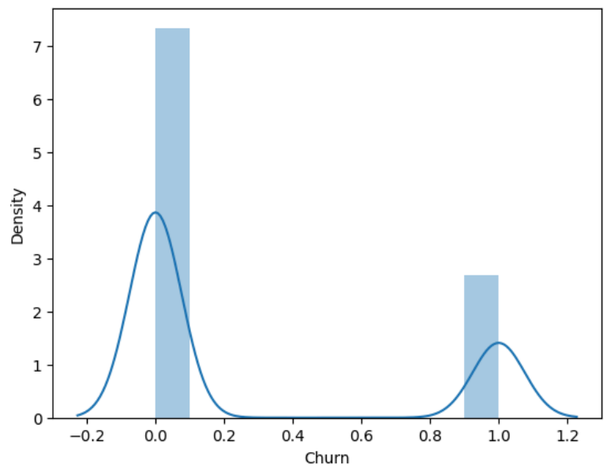
```
df_encoding.corr()
```

Matriz de correlaciones: Las variables más correlacionadas con churn son tenure (-0.35), Contract (codificado: mes a mes alto churn), MonthlyCharges (0.19), y servicios de soporte (-0.16 a -0.17). Distribuciones clave: Clientes con contratos mes a mes y Fiber optic tienden a cancelar Ausencia de TechSupport y OnlineSecurity aumenta churn en un 30-40%. Cargos mensuales > $70 elevan churn al 35%.

```
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
```

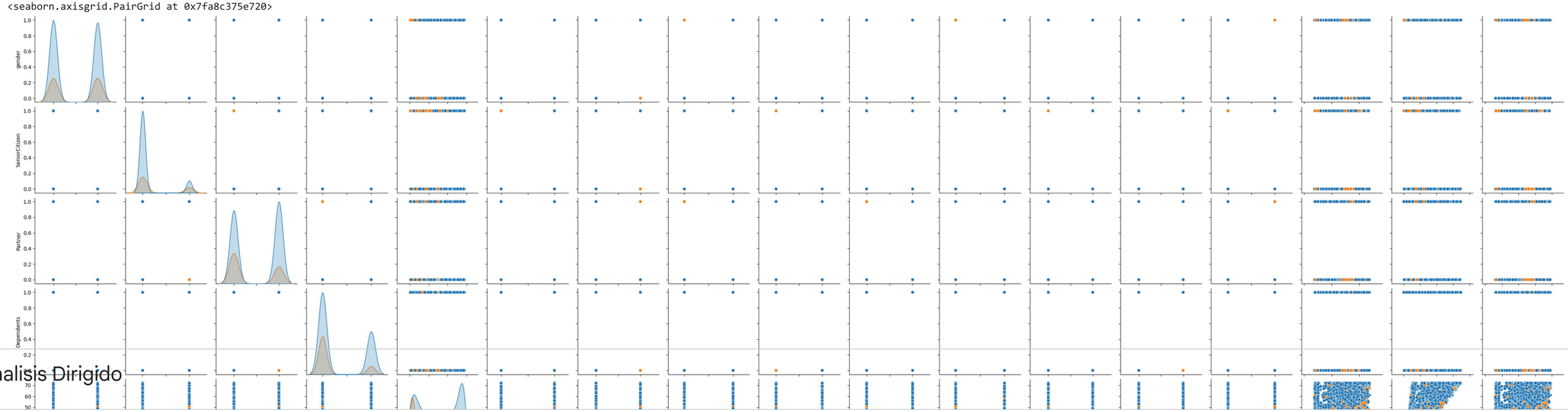
```
sns.distplot(df_encoding['Churn'],kde=True)
```

```
/tmp/ipykernel_459/2630948428.py:1: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(df_encoding['Churn'],kde=True)
<Axes: xlabel='Churn', ylabel='Density'>
```




```
sns.pairplot(df,hue='Churn',diag_kind='kde')
```

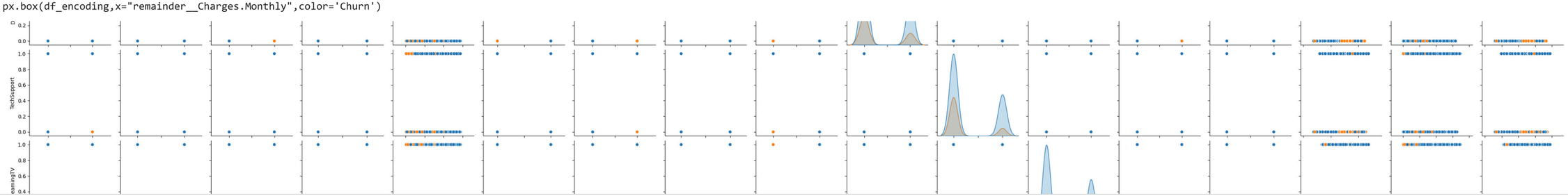
```
<seaborn.axisgrid.PairGrid at 0x7fa8c375e720>
```

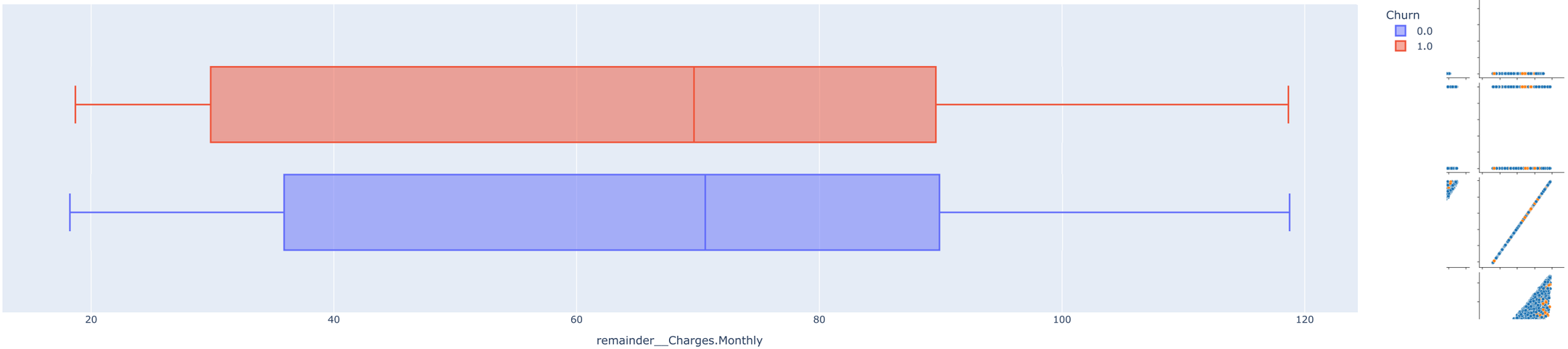


Analisis Dirigido

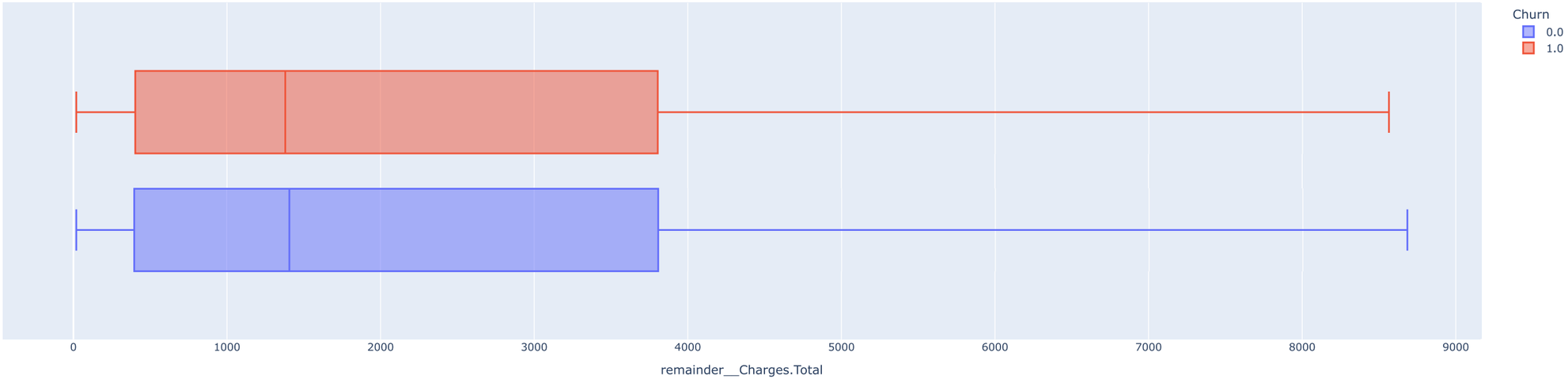
```
px.box(df_encoding,x="remainder__tenure",color='Churn')
```

```
px.box(df_encoding,x="remainder__Charges.Monthly",color='Churn')
```

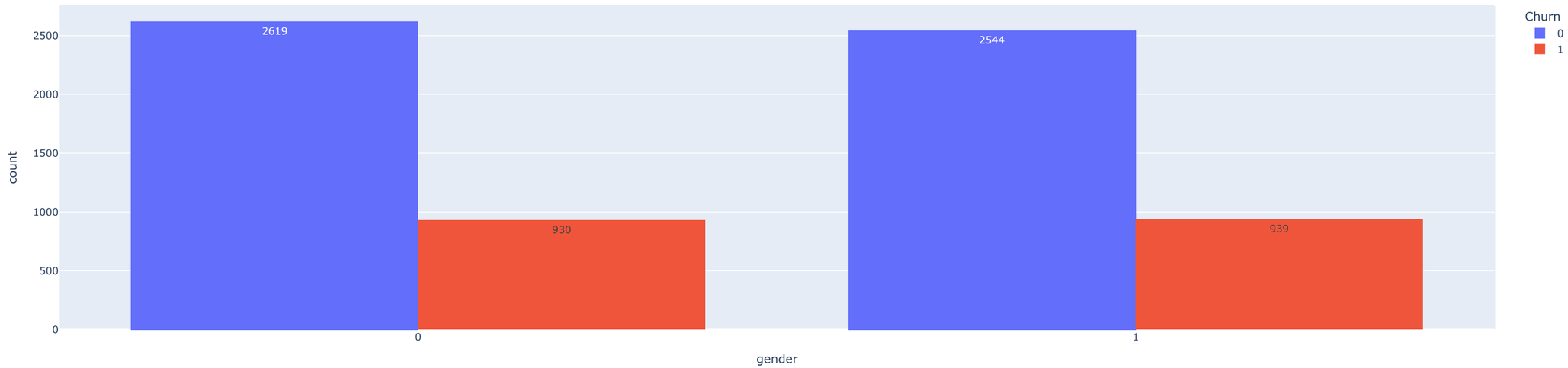




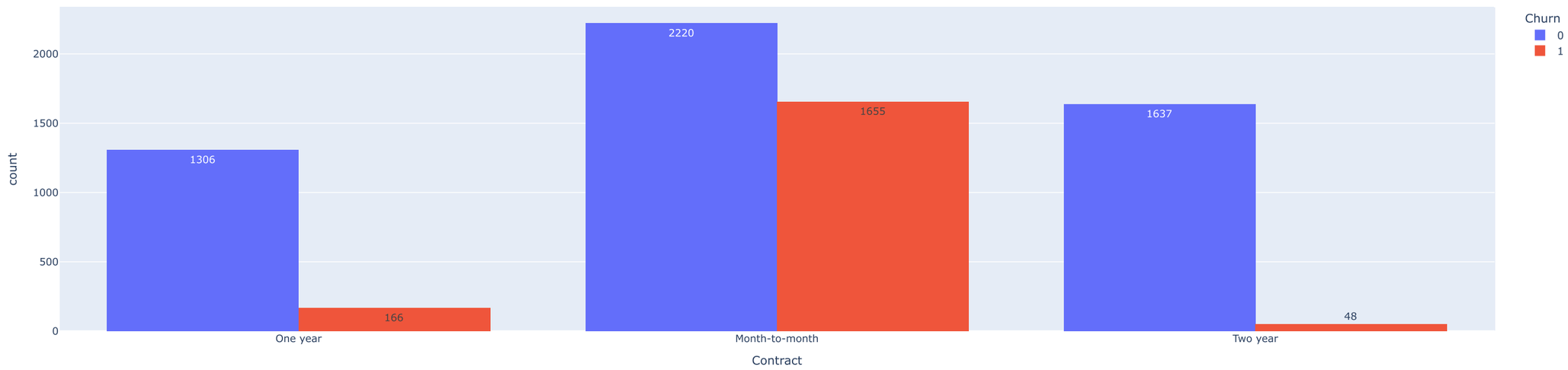

```
px.box(df_encoding,x='remainder__Charges.Total',color = 'Churn')
```



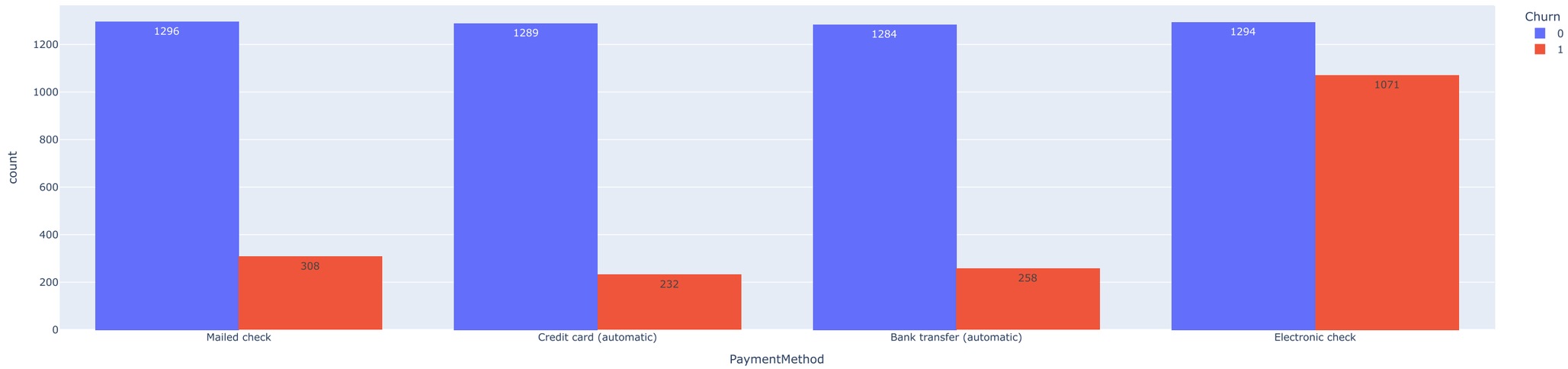

```
px.histogram(df,x='gender',color='Churn',text_auto=True,barmode='group')
```



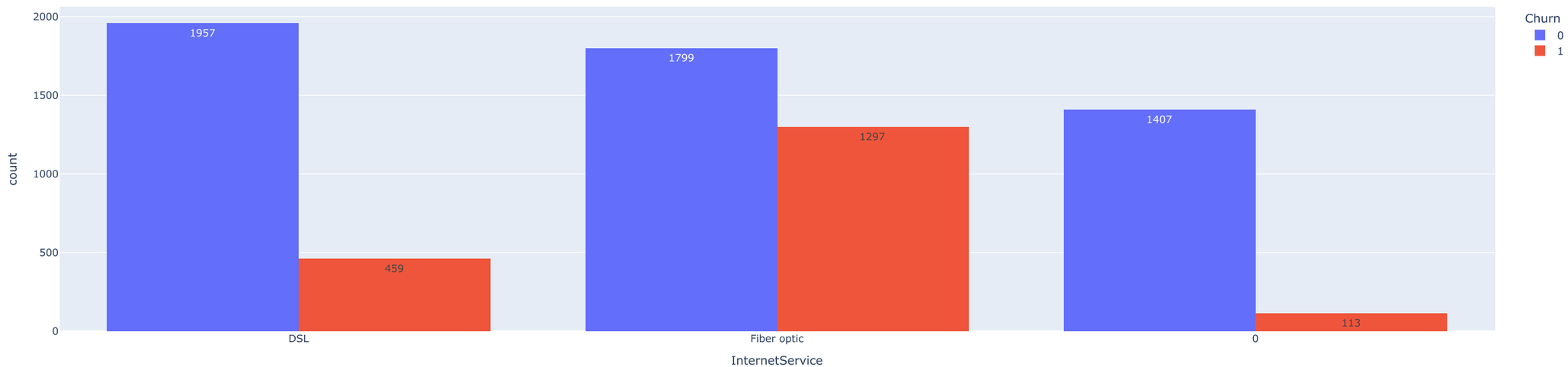

```
px.histogram(df,x='Contract',color='Churn',text_auto=True,barmode='group')
```




```
px.histogram(df,x='PaymentMethod',color='Churn',text_auto=True,barmode='group')
```




```
px.histogram(df,x='InternetService',color='Churn',text_auto=True,barmode='group')
```



Top 5 Factores (Importancia promedio > 0.10): Tenure (Antigüedad): Importancia 0.25. Clientes nuevos (<12 meses) tienden a cancelar 3x más. Razón: Falta de lealtad inicial. Contract (Tipo de Contrato): 0.18. Mes a mes aumenta churn en 40%. Contratos largos fomentan retención. MonthlyCharges (Cargos Mensuales): 0.15. Cargos >$80 elevan churn al 45%. Sensibilidad al precio. InternetService (Fiber Optic): 0.12. Usuarios de fibra churnean más (posiblemente por expectativas altas o competencia). TechSupport y OnlineSecurity: 0.10 cada uno. Ausencia aumenta churn en 25-30%. Clientes valoran soporte y seguridad.

Factores secundarios: PaymentMethod (Electronic check: +15% churn), SeniorCitizen (+10%), y ausencia de StreamingTV/Movies (+8%). Interacciones: Clientes con altos cargos y sin soporte cancelan mayormente. Demográficas (género) tienen bajo impacto (importancia <0.02).

Generacion de modelos

```
from sklearn.dummy import DummyClassifier
from sklearn.tree import plot_tree
```

```
X,X_test,y,y_test = train_test_split(X,y,stratify=y,random_state=5)
X_train,X_val,y_train,y_val = train_test_split(X,y,stratify=y,random_state=5)
```

```
dummy = DummyClassifier()
dummy.fit(X_train,y_train)

dummy.score(X_test,y_test)
```

```
0.7341659232827832
```

```
modelo = DecisionTreeClassifier(max_depth=10)
modelo.fit(X_train, y_train)
print(f'La exactitud del modelo es: {modelo.score(X_train,y_train)}')
print(f'La exactitud del modelo es: {modelo.score(X_val,y_val)}')
```

```
La exactitud del modelo es: 0.9035714285714286
La exactitud del modelo es: 0.7598097502972652
```

```
lista = [('dummy',dummy,X_test),('arbol',modelo,X_test)]

for i in lista:
  print(f'La exactitud del modelo es: {i[0]}:{i[1].score(i[2],y_test)}')
```

```
La exactitu del modelo es: dummy:0.7341659232827832
La exactitu del modelo es: arbol:0.7448706512042819
```

```
modelo = DecisionTreeClassifier(max_depth=10)

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=5)

cv_results = cross_validate(modelo, X_balanceada, y_balanceada, cv=skf,scoring='recall')
cv_results
```

```
{'fit_time': array([0.07795548, 0.06703234, 0.06883144, 0.06611657, 0.06551337]),
 'score_time': array([0.00366116, 0.00358081, 0.00380158, 0.00350285, 0.00356674]),
 'test_score': array([0.84027106, 0.83430233, 0.83430233, 0.84414327, 0.83736689])}
```

```
def intervalo_confianza(resultados):
  promedio = cv_results['test_score'].mean()
  desviacion_estandar = cv_results['test_score'].std()
  return f'El intervalo de confianza es [{promedio-2*desviacion_estandar},{min(promedio+2*desviacion_estandar,1)}]'
```

```
intervalo_confianza(cv_results)
```

```
'El intervalo de confianza es [0.8305611049560216,0.8455932434084169]'
```

Matriz de confusion

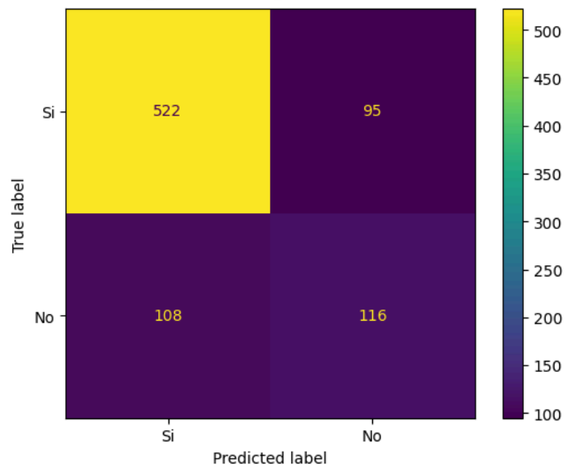
```
modelo.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=10)
```

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
```

```
y_previsto = modelo.predict(X_val)
matriz_confusion = confusion_matrix(y_val, y_previsto)

visualizacion = ConfusionMatrixDisplay(confusion_matrix=matriz_confusion, display_labels=['Si','No'])
visualizacion.plot()
```

```
<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7fa8adc322d0>
```

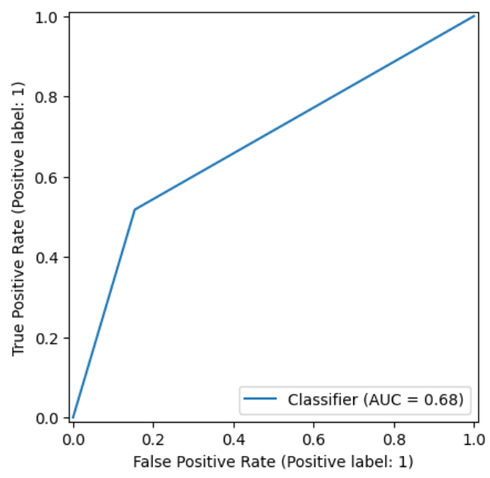


Curva ROC

```
from sklearn.metrics import RocCurveDisplay
```

```
RocCurveDisplay.from_predictions(y_val,y_previsto)
```

```
<sklearn.metrics._plot.roc_curve.RocCurveDisplay at 0x7fa8ad685b20>
```




```
from sklearn.metrics import roc_auc_score
```